

บทที่ 3

ทฤษฎีและหลักการของโปรแกรมที่เกี่ยวข้อง

3.1 การเขียนโปรแกรมให้ทำงานเป็นมัลติทาสก์

เป็นโหมมคการทำงานที่อนุญาตให้สามารถทำงานพร้อมๆกันได้ หรือ ประมวลผลแบบสอดแทรกกันของสองทาสก์ หรือมากกว่า คล้ายกับมัลติโปรแกรมมิ่งที่มีรูปแบบแตกต่างกัน

ในระบบปฏิบัติการคอมพิวเตอร์มีข้อจำกัดคือไม่อนุญาตโปรแกรมหลายๆ โปรแกรมทำงานพร้อมๆกันต้องทำงานให้เสร็จทีละงานเป็นลำดับไป ดังนั้น งานในการจัดการให้แต่ละโปรแกรมหรือหลายๆงานทำไปพร้อมกันจึงเป็นของผู้สร้างโปรแกรม

ในปัจจุบันมีไลบรารีที่มาช่วยจัดการให้โปรแกรมหนึ่งๆ สามารถรันหลายๆฟังก์ชันสลับกันไปมาได้หลายตัวด้วยกัน อาทิเช่น ไมโครซีไอเอส (อ่านว่า MicroC/OS), Multi-C library เป็นต้น ซึ่งในโครงการนี้ผู้พัฒนาเลือกใช้เนื่องจากไม่ขึ้นกับหน่วยประมวลผล และสนับสนุนการทำงานแบบเรียลไทม์ (real time)

3.2 ไมโครคอนโทรลเลอร์ โอเพอร์เรตติ้ง ซิสเต็มเวอร์ชัน 2

ไมโครคอนโทรลเลอร์ โอเพอร์เรตติ้ง ซิสเต็มเวอร์ชัน 2 หรือไมโครซีไอเอส2 (μC/OS-II) ถูกสร้างขึ้นบนพื้นฐานของเรียลไทม์เคอร์เนล (Real-time kernel) ซึ่งนิยมนำไปใช้ในอุปกรณ์หลายๆอย่างด้วยกัน เช่น กล้องถ่ายรูป เน็ตเวิร์คอะแดปเตอร์ อุตสาหกรรม เป็นต้น

3.2.1 จุดเด่นของไมโครคอนโทรลเลอร์ โอเพอร์เรตติ้ง ซิสเต็มเวอร์ชัน 2

1. ซอร์สโค้ด (Source Code) มีการจัดระเบียบไว้เป็นอย่างดีและเข้าใจง่าย
2. พอร์แทเบิล (Portable) โดยส่วนใหญ่แล้วไมโครซีไอเอส2 จะถูกเขียนโดย ANSI C ที่มีความสามารถในการพอร์แทเบิลสูง ด้วยการเขียนด้วยแอสเอ็ม (ASM) ที่ทำให้ไมโครซีไอเอส2 มีขนาดเล็ก และสามารถพอร์ตไปยังชิปประมวลผลตัวอื่น ๆ ได้เช่นเดียวกันกับ ไมโครซีไอเอส ไมโครซีไอเอส2 สามารถถูกพอร์ตลงไปในชิปประมวลผลตัวที่ใหญ่กว่าได้ ครอบคลุมที่ชิปประมวลผลนั้น ๆ มีสแต็กพอยเตอร์ (stack pointer) และซีพียู (CPU register) ที่สามารถถูกพุช (push) และป๊อป (pop) ลงไปในสแต็กได้

พอร์ตทั้งหมดที่มีอยู่ในไมโครซีไอเอสสามารถถูกแปลงมาลงในไมโครซีไอเอส2 ภายในเวลาประมาณ 1 ชม. เพราะไมโครซีไอเอส2 สามารถเข้ากันได้กับไมโครซีไอเอส ทำให้โปรแกรมประยุกต์ของไมโครซีไอเอสสามารถรันบนไมโครซีไอเอส2 ได้ด้วยการเปลี่ยนแปลงที่เล็กน้อย หรือว่าไม่เปลี่ยนแปลงเลย สามารถตรวจสอบพอร์ตที่มีอยู่ในปัจจุบันของไมโครซีไอเอส2 ได้จาก เว็บไซต์ www.uCOS-II.com

3. รอมเมเบิล (ROMable) ไมโครซีไอเอส2ได้ถูกออกแบบสำหรับโปรแกรมประยุกต์ฝังตัว หมายความว่าถ้าเรามี proper tool chain (เช่น C compiler, assembler, และ linker/loader) คุณ สามารถฝัง μC/OS-II ลงไปในส่วนของชิปงานของคุณได้

4. แสเกลเอเบิล (Scalable) ผู้สร้างได้ออกแบบไมโครซีไอเอส2ในแบบที่เราสามารถใช้เฉพาะบริการที่เราต้องการในงานได้ หมายถึงงานของเราสามารถใช้งานบริการบนไมโครซีไอเอส2บางส่วน จนถึง

ใช้งานบริการทุกส่วน และยอมให้คุณลดขนาดของหน่วยความจำ (ทั้ง ROM และ RAM) ที่ถูกใช้งานโดยไมโครซีไอเอส2 บนแต่ละงานได้

5. 프리เอมทิฟ (Preemptive) ไมโครซีไอเอส2เป็น “fully preemptive real-time kernel” ซึ่งหมายความว่า ไมโครซีไอเอส2จะทำงานงานที่มีความสำคัญสูงที่สุดที่พร้อมทำงานก่อนเสมอ ซึ่งโดยเคอร์เนลตามท้องตลาดส่วนใหญ่แล้วจะเป็นแบบฟรีเอมทิฟและไมโครซีไอเอส2เทียบเคียงได้กับเคอร์เนลเหล่านั้น

6. มัลติทาสกิ้ง (Multitasking) ไมโครซีไอเอส2สามารถจัดการทาสก์ได้ถึง 64 ทาสก์พร้อมกัน อย่างไรก็ตามซอฟต์แวร์รุ่นปัจจุบันได้จองทาสก์ไว้ 8 ทาสก์เพื่อให้สำหรับระบบใช้งาน ทำให้เหลือเพียง 56 ทาสก์เท่านั้น แต่ละทาสก์จะมีเลขกำกับที่ไม่ซ้ำกัน ซึ่งหมายความว่าไมโครซีไอเอส2ไม่สามารถที่จะทำ round-robin scheduling ได้เหตุนี้ทำให้มี 64 ระดับความสำคัญ (priority level)

7. ดีเทอร์มินิสติก (Deterministic) เวลาในการประมวลผลของไมโครซีไอเอส2สามารถวัดได้ นั่นหมายถึงเราสามารถวัดรู้และคำนวณเวลาที่ไมโครซีไอเอส2ใช้ในการคำนวณฟังก์ชันและบริการ แต่อย่างไรก็ตามเวลาในการประมวลผลของไมโครซีไอเอส2เซอร์วิสทั้งหมดไม่ขึ้นอยู่กับจำนวนของทาสก์ในงานของเรา

8. ทาสก์สแต็ค (Task Stacks) แต่ละทาสก์ต้องการสแต็คของตัวเอง อย่างไรก็ตามไมโครซีไอเอส2ยอมให้แต่ละทาสก์มีขนาดของสแต็คที่แตกต่างกันได้ ซึ่งสามารถช่วยให้เราลดขนาดการใช้งานบนแรมบนงานของเราได้ด้วย stack-checking ของไมโครซีไอเอส2 เราสามารถประมาณค่าของสแต็คที่เราต้องการใช้บนแต่ละทาสก์ของเราได้อย่างเท่าที่การใช้งานจริง ๆ

9. บริการ (Services) ไมโครซีไอเอส2 ได้เตรียมบริการต่าง ๆ เช่น mailboxes queues semaphores partition เป็นต้น

10. การจัดการอินเทอร์รัพท์ (Interrupt Management) อินเทอร์รัพท์สามารถหยุดการทำงานของทาสก์ไว้ชั่วคราวได้ โดยที่ทาสก์ที่มีความสำคัญที่สูงกว่าจะสามารถคั่นขึ้นมาทำงานระหว่างอินเทอร์รัพท์ของทาสก์ที่มีความสำคัญต่ำกว่าได้ โดยที่อินเทอร์รัพท์สามารถถูกใช้ได้ถึง 255 ระดับซ้อนกัน

11. มีความสามารถมากและเชื่อถือได้ (Robust and Reliable) ไมโครซีไอเอส2มีพื้นฐานอยู่บนไมโครซีไอเอส ซึ่งถูกใช้ในหลายร้อยงานทั่วโลกมาตั้งแต่ปี 1992 ซึ่งไมโครซีไอเอส2ใช้คอร์ (core) เดิมและฟังก์ชันที่ยังคงอยู่เหมือนเดิมเป็นส่วนใหญ่

3.2.2 แนวคิดแบบเรียลไทม์ซิสเต็ม

เรียลไทม์ซิสเต็มได้ถูกกำหนดลักษณะโดยผลที่ตามมาซึ่งถูกบังคับอันเนื่องมาจากทางตรรกะ เช่นเดียวกันกับคุณสมบัติความถูกต้องของเวลาของระบบที่ไม่ได้เป็นดังนั้น แบ่งออกเป็น 2 ชนิดคือ ซอฟท์ (SOFT) และฮาร์ด (HARD) สำหรับ SOFT real-time system นั้นทาสก์จะถูกกระทำโดยระบบเร็วที่สุดครบเท่าที่จะทำได้ แต่ทาสก์อาจจะไม่เสร็จได้ทันตามเวลาที่กำหนดไว้ สำหรับแบบ Hard real-time system จะถูกกำหนดให้งานออกมาถูกต้องและเสร็จตามเวลาที่กำหนด เรียลไทม์ซิสเต็มโดยส่วนใหญ่แล้วจะประกอบไปด้วยทั้งแบบ SOFT และ HARD real-time application นั้นมีขอบคลุมกว้างมาก แต่เรียลไทม์ซิสเต็มส่วนใหญ่นั้นจะเป็นแบบฝังตัว หมายความว่าคอมพิวเตอร์ที่ถูกสร้างในระบบจะไม่ถูกผู้ใช้เห็น โดยที่เรียลไทม์ซิสเต็มนั้นออกแบบได้ยากกว่าระบบที่ไม่เรียลไทม์ซิสเต็มมาก

3.2.3 มิวเชอร์ลเอ็กซ์คลูชัน (Mutual Exclusion)

วิธีการที่ง่ายที่สุดสำหรับทาสก์ที่จะติดต่อสื่อสารกันผ่านข้อมูลที่ใช้ร่วมกัน เมื่อมีการแลกเปลี่ยนข้อมูลระหว่างกันบนข้อมูลที่ใช้ร่วมกัน เราต้องมั่นใจว่าแต่ละทาสก์จะมีการเข้าถึงข้อมูลร่วมกันโดยหลีกเลี่ยงการแย่งชิงข้อมูล โดยวิธีการโดยทั่วไปที่จะเข้าถึงข้อมูลร่วมกันมีดังนี้

- ไม่ให้ใช้อินเตอร์รัพท์ (Disable interrupts)
- ทำเทสต์และเซตโอเปอร์เรชัน (Performing test-and-set operations)
- ไม่ให้มีการใช้การจัดลำดับ (Disabling scheduling)
- ใช้เซมาฟอร์ (Using semaphores)

3.2.4 เซมาฟอร์ (Semaphores)

เป็นโปรโตคอลสำหรับมัลติทาสก์กิงเคอร์เนล (Multitasking kernels) โดยส่วนใหญ่ มีจุดประสงค์เพื่อ

- ควบคุมการเข้าถึงทรัพยากรร่วม
- ส่งสัญญาณของเหตุการณ์ที่เกิดขึ้น
- ยอมให้ 2 ทาสก์ทำการซิงค์ (Sync) งานกันได้

เซมาฟอร์เป็นกฎที่จะทำให้โค้ดของเราเรียงลำดับการทำงาน ถ้าเซมาฟอร์กำลังถูกใช้งานอยู่ทาสก์ที่เรียกเข้ามาจะถูกพักไว้ชั่วคราวจนกระทั่งเซมาฟอร์ถูกปล่อยจากเจ้าของปัจจุบัน โดยแบ่งเซมาฟอร์ได้เป็น 2 ชนิด คือ Binary semaphore และ counting semaphore ดังเช่นที่ชื่อได้แสดงไว้ binary semaphore สามารถมีค่าได้เพียงแค่ 0 หรือ 1 ส่วน counting semaphore สามารถมีค่าได้ตั้งแต่ 0 ถึง 255, 65535, 4294967295 แล้วแต่โครงสร้างของแต่ละเซมาฟอร์ที่เขียนโดยใช้ 8, 16, 32 บิต แต่ขนาดจริงๆ จะขึ้นอยู่กับที่เคอร์เนลที่ใช้ เมื่อมีการนำเซมาฟอร์มาใช้งาน ตัวระบบเองต้องมีการเก็บเส้นทางของทาสก์ที่ทำการรอเพื่อให้เซมาฟอร์ดำเนินการได้

โดยปกติแล้ว มีเพียงแค่ 3 โอเปอเรชัน ที่ถูกเตรียมไว้ใช้ในเซมาฟอร์ โดยประกอบด้วย INITIALIZE (หรือเรียกว่า CREATE), WAIT (หรือเรียกว่า PEND), SIGNAL (หรือเรียกว่า POST) สำหรับค่าเริ่มต้นของเซมาฟอร์ต้องถูกเตรียมในช่วงที่เซมาฟอร์ถูก initialized และทาสก์ที่รออยู่ของเซมาฟอร์จะต้องถูกทำให้ว่าง

ทาสก์ที่ต้องการเซมาฟอร์จะเรียก WAIT แล้วถ้าเซมาฟอร์นั้นสามารถใช้งานได้ (ค่ามีค่าเกิน 0) ค่าของเซมาฟอร์จะถูกลดค่าลงและทาสก์จะสามารถทำงานได้ต่อไป แต่ถ้าค่าของเซมาฟอร์มีค่าเป็น 0 แล้วทาสก์มีการเรียก WAIT จะถูกทำการถูกนำไปไว้ใน waiting list โดยเคอร์เนล ส่วนใหญ่จะยอมให้กำหนด timeout เพื่อกำหนดเวลาที่รอเซมาฟอร์ โดยจะมีการเรียกทาสก์ขึ้นมารันและแสดงข้อผิดพลาดกลับไปยังผู้ที่เรียกทาสก์นั้น ทาสก์ที่ปล่อย เซมาฟอร์ออกมาจะไปเรียก SIGNAL ถ้าไม่มีทาสก์อื่นรออยู่ใน waiting list ค่าของเซมาฟอร์ก็จะเพิ่มขึ้น แต่ถ้ามีทาสก์เหลืออยู่ในลิสต์จะทำการเรียกทาสก์ในลิสต์ที่ขึ้นมารันโดยไม่มีที่เพิ่มค่าเซมาฟอร์ สำหรับเงื่อนไขในการหยิบทาสก์ขึ้นมาทำงานนั้น ขึ้นอยู่กับเคอร์เนลดังเช่น

- ทาสก์ที่มีระดับความสำคัญสูงสุดที่รออยู่
- ทาสก์แรกที่เข้ามารอในลิสต์ (First In First Out หรือ FIFO)

3.2.5 การสื่อสารระหว่างทาสก์ (Intertask Communication)

บางครั้งก็เป็นความจำเป็นของทาสก์หรือไอเอสอาร์ (ISR) ที่จะติดต่อข้อมูลกับทาสก์อื่น ๆ ข้อมูลที่แลกเปลี่ยนเรียกการสื่อสารระหว่างทาสก์ โดยข้อมูลที่ถูกใช้ในการสื่อสารระหว่างทาสก์ทำได้ 2 ทาง คือ ผ่านตัวแปรกลาง หรือ ส่งข้อความ

เมื่อใช้การผ่านตัวแปรกลาง แต่ละทาสก์หรือไอเอสอาร์ต้องแน่ใจว่ามีการเข้าถึงข้อมูลเพียงทีละหนึ่งทาสก์เท่านั้น ถ้าไอเอสอาร์ได้ถูกเรียกแล้วมีทางเดียวที่จะเข้าถึงข้อมูลได้อย่างปลอดภัยคือ การไม่ให้ใช้อินเทอร์รัพท์ (disabled interrupt) แต่ถ้ามี 2 ทาสก์ที่ขอใช้งานตัวแปรร่วมตัวเดียวกันสามารถทำได้โดยไม่ให้อินเทอร์รัพท์หรือเรียกใช้เซมาฟอร์

ทาสก์สามารถติดต่อกับไอเอสอาร์ได้เพียงเดียวเท่านั้นคือผ่านตัวแปรกลาง ซึ่งโดยที่จริงแล้วทาสก์ไม่จำเป็นต้องกลัวว่าไอเอสอาร์จะไปเปลี่ยนค่าในตัวแปรกลาง จึงไม่มีความจำเป็นที่ไอเอสอาร์จะต้องส่งสัญญาณไปยังทาสก์ โดยใช้เซทาฟอร์หรือโพล (Poll) เพื่อให้เหมาะกับสถานการณ์เช่นนี้ คุณควรเลือกใช้เมสเสจบ็อกซ์ (message mailbox)

3.2.6 เมสเสจบ็อกซ์

ข้อความสามารถถูกส่งไปยังทาสก์ผ่าน Kernel services คือเมสเสจบ็อกซ์หรือเรียกว่า Message exchange โดยเก็บผ่านตัวแปรพอยเตอร์ โดยทาสก์หรือไอเอสอาร์สามารถฝากเมสเสจลงในเมสเสจบ็อกซ์และ 1 หรือ หลายทาสก์สามารถรับเมสเสจผ่านบริการที่เคอร์เนลเตรียมไว้ โดยการส่งและรับข้อความจะเป็นการส่งข้อมูลที่พอยเตอร์นั้น ๆ ซี่อยู่

Waiting list ที่สัมพันธ์แต่ละเมสเสจบ็อกซ์ ในกรณีที่มี 1 หรือ หลาย ๆ ทาสก์ต้องการรับข้อความผ่านเมสเสจบ็อกซ์ โดยทาสก์ที่ต้องการข้อความจากเมสเสจบ็อกซ์ที่ว่าง จะถูกทำให้พักการทำงานและถูกทำให้รออยู่ใน Waiting list จนกระทั่งมีข้อความเข้ามาในเมสเสจบ็อกซ์ โดยเคอร์เนลอนุญาตให้รอได้จนถึง timeout ถ้าข้อความยังไม่ถึงก่อนเวลาที่กำหนดไว้ทาสก์ที่รอรับข้อความจะถูกทำให้ ready และทำงาน โดยส่ง error code กลับไป เมื่อข้อความได้ถูกนำออกจากเมสเสจบ็อกซ์ทาสก์ที่มีระดับความสำคัญสูงที่สุดจะทำการรับข้อความ (แบบ priority based) หรือทาสก์แรกที่เข้ามารอเป็นผู้รับข้อความ (แบบ FIFO based)

เคอร์เนลได้เตรียมบริการสำหรับเมสเสจบ็อกซ์ดังนี้

- กำหนดค่าเริ่มต้นของเนื้อหาในเมสเสจบ็อกซ์ในตอนแรกเริ่มเมสเสจบ็อกซ์สามารถกำหนดให้มี หรือ ไม่มีเมสเสจได้
- ฝากเมสเสจลงในเมสเสจบ็อกซ์ (POST)
- รอรับเมสเสจที่ได้ทำการฝากลงในเมสเสจบ็อกซ์ (PEND)
- รับเมสเสจจากเมสเสจบ็อกซ์ ถ้ามีข้อความอยู่ แต่จะไม่พักการทำงานของผู้ที่เรียกถ้าเมสเสจบ็อกซ์ ว่างเปล่า (ACCEPT) ถ้าเมสเสจบ็อกซ์มีข้อความอยู่ ข้อความจะถูกนำออกจากเมสเสจบ็อกซ์ และส่งโค้ดกลับจะถูกใช้นำไปใช้ในการเตือนผู้ที่เรียกเกี่ยวกับผลลัพธ์ของการเรียก